



UNIVERSIDAD AGRARIA DEL ECUADOR

Facultad de Ciencias Agrarias

Carrera Ingeniería en Ciencias de la Computación — Modalidad en Línea

Proyecto Final — Avance Documental (50%)

Sistema de Gestión de Biblioteca Universitaria

Asignatura: Lenguaje de Programación 2 (Programación 2)

Docente: Ing. Bryan Orlando Vélez San Martín, M.Sc.

Paralelo: 3SA — Tercer Semestre

Grupo: Grupo #1

Integrantes:

- Henry Josué Pazmiño Raimundy
- Jodie Michelle Parrales Quimí
- Cindy Ninoska Ayoví Gonzabay
- Cesar Vicente Hernández Uyaguari
- Mayra Alejandra Vera Yagual

Julio 2026

Índice de Contenidos (Preliminar)

1. Portada
2. Índice de Contenidos
3. Introducción
4. Justificación
5. Objetivos
 - 5.1 Objetivo General
 - 5.2 Objetivos Específicos
6. Alcance del Sistema
7. Marco Teórico
 - 7.1 Programación Orientada a Objetos
 - 7.2 Estructuras de Datos
 - 7.3 Algoritmos de Ordenamiento y Búsqueda
 - 7.4 Base de Datos con SQLite
8. Diseño Preliminar del Sistema
 - 8.1 Diagrama de Clases UML
 - 8.2 Modelo Entidad-Relación de la Base de Datos

Introducción

El presente documento constituye el avance del 50% del proyecto final de la asignatura Lenguaje de Programación 2, correspondiente al Tercer Semestre de la Carrera de Ciencias de la Computación de la Universidad Agraria del Ecuador. El proyecto consiste en el desarrollo de un Sistema de Gestión de Biblioteca Universitaria, una aplicación de escritorio desarrollada en Python que integra los conceptos fundamentales de programación orientada a objetos, estructuras de datos, algoritmos de ordenamiento y búsqueda, bases de datos relacionales con SQLite, e interfaz gráfica con Tkinter.

La biblioteca universitaria enfrenta desafíos comunes en la gestión de sus recursos bibliográficos: control de inventario de libros, registro de socios (estudiantes y docentes), administración de préstamos y devoluciones, y manejo de reservas de ejemplares. El sistema propuesto automatiza estos procesos mediante una solución informática integral que reemplaza los procedimientos manuales y reduce el riesgo de pérdida de información.

Este informe cubre las secciones de introducción, justificación, objetivos, alcance, marco teórico y diseño preliminar del sistema. La segunda mitad del documento (metodología detallada, implementación, pruebas, análisis de complejidad, conclusiones y bibliografía) se completará en la entrega final de la semana 14.

Justificación

El desarrollo del Sistema de Gestión de Biblioteca Universitaria se justifica desde una perspectiva académica y práctica. Desde el punto de vista académico, este proyecto integra la totalidad de los contenidos vistos durante el semestre: los principios de la programación orientada a objetos (herencia, encapsulamiento, polimorfismo y abstracción), las estructuras de datos lineales (listas enlazadas, pilas y colas), los algoritmos fundamentales de ordenamiento y búsqueda, y el manejo de bases de datos relacionales mediante SQLite.

En el ámbito práctico, una biblioteca universitaria típica maneja cientos o miles de ejemplares, atiende a una comunidad de estudiantes y docentes, y requiere llevar un control preciso de los préstamos, devoluciones y reservas. Un sistema informático permite agilizar estos procesos, reducir errores humanos, mantener un historial confiable de movimientos y facilitar la consulta del catálogo. El proyecto, aunque desarrollado como caso simulado, responde a una necesidad real identificada en instituciones educativas.

Además, el proyecto fomenta el trabajo en equipo, la división de responsabilidades y el uso de herramientas de control de versiones como GitHub, preparando al grupo para enfrentar proyectos de software en entornos profesionales.

Objetivos

5.1 Objetivo General

Desarrollar un Sistema de Gestión de Biblioteca Universitaria funcional, implementado en Python, que integre los conceptos de programación orientada a objetos, estructuras de datos, algoritmos de ordenamiento y búsqueda, base de datos SQLite e interfaz gráfica con Tkinter, como proyecto integrador de la asignatura Lenguaje de Programación 2.

5.2 Objetivos Específicos

- Aplicar los principios de POO (herencia, encapsulamiento, polimorfismo y abstracción) en el modelado de las entidades del sistema: Persona, Estudiante, Docente, Socio, Libro, Préstamo y Reserva.
- Implementar estructuras de datos lineales como lista enlazada simple para el catálogo de libros, cola FIFO para la gestión de reservas y pila LIFO para la funcionalidad de deshacer préstamos.
- Incorporar algoritmos de ordenamiento (burbuja, inserción, merge sort y quick sort) y búsqueda (lineal y binaria) para la organización y consulta del catálogo.
- Diseñar e implementar una base de datos relacional en SQLite con al menos tres tablas relacionadas, incluyendo operaciones CRUD completas.
- Desarrollar una interfaz gráfica con Tkinter que permita a los usuarios realizar las operaciones principales del sistema de forma intuitiva.

Alcance del Sistema

El sistema incluye: registro y gestión de socios (estudiantes y docentes), administración del catálogo de libros con altas, bajas y modificaciones, registro de préstamos y devoluciones con control de fechas, gestión de reservas de ejemplares

mediante cola de espera, consulta del catálogo con filtros por título, autor o categoría, ordenamiento del catálogo por distintos criterios (título, autor, año), y generación de reportes básicos de préstamos activos.

El sistema no incluye: integración con sistemas externos (SIRBI, sistemas contables), módulo de multas o sanciones, acceso web o remoto, notificaciones por correo electrónico, ni soporte para múltiples sedes o sucursales.

Marco Teórico

7.1 Programación Orientada a Objetos

La programación orientada a objetos (POO) es un paradigma de programación que organiza el código en torno a objetos que contienen datos (atributos) y métodos (funciones) que operan sobre esos datos. Sus cuatro pilares fundamentales son:

- **Herencia:** Permite que una clase derive de otra, heredando sus atributos y métodos. En el proyecto, Estudiante y Docente heredan de Persona, reutilizando atributos comunes como nombre, identificación y teléfono.
- **Encapsulamiento:** Oculta los detalles internos de una clase, exponiendo solo una interfaz controlada mediante getters y setters. Se implementa con atributos privados (doble guion bajo `__`) en las clases del proyecto.
- **Polimorfismo:** Permite que objetos de diferentes clases respondan al mismo método de forma particular. La clase Socio actúa como envoltorio (wrapper) que recibe una Persona (Estudiante o Docente) y delega polimórficamente el cálculo de límites de préstamo.
- **Abstracción:** Oculta la complejidad interna y muestra solo la funcionalidad esencial. La clase Persona se define como abstracta, declarando métodos como `tipo_socio()` que las subclases deben implementar.

7.2 Estructuras de Datos

Las estructuras de datos lineales implementadas en el proyecto son:

Lista enlazada simple: Estructura compuesta por nodos, donde cada nodo contiene un dato y una referencia al siguiente nodo. Se utiliza para el catálogo dinámico de libros, permitiendo inserción y eliminación sin necesidad de redimensionar arreglos.

Cola FIFO (First In, First Out): Estructura donde el primer elemento en ingresar es el primero en salir. Se implementa para gestionar las reservas de libros, asegurando que los socios sean atendidos en orden de solicitud.

Pila LIFO (Last In, First Out): Estructura donde el último elemento en ingresar es el primero en salir. Se utiliza para la funcionalidad de deshacer préstamos recientes, permitiendo revertir la última operación realizada.

7.3 Algoritmos de Ordenamiento y Búsqueda

El proyecto implementa los siguientes algoritmos:

- **Ordenamiento burbuja (Bubble Sort):** compara pares de elementos adyacentes y los intercambia si están en el orden incorrecto. Repite el proceso hasta que la lista está ordenada.
- **Ordenamiento por inserción (Insertion Sort):** construye la lista ordenada tomando elementos uno por uno e insertándolos en la posición correcta.
- **Merge Sort:** algoritmo divide y vencerás que divide la lista en mitades, las ordena recursivamente y las fusiona.
- **Quick Sort:** selecciona un pivote, particiona la lista en elementos menores y mayores que el pivote, y ordena recursivamente cada partición.
- **Búsqueda lineal:** recorre la lista elemento por elemento hasta encontrar el valor buscado.
- **Búsqueda binaria:** requiere una lista ordenada y divide repetidamente el espacio de búsqueda a la mitad hasta encontrar el elemento.

7.4 Base de Datos con SQLite

SQLite es un sistema de gestión de bases de datos relacional embebido, que no requiere servidor y almacena toda la información en un único archivo .db. Se utiliza en el proyecto para persistir los datos de socios, libros, préstamos y reservas, utilizando el módulo `sqlite3` de Python. Las operaciones CRUD (Create, Read, Update, Delete) se implementan siguiendo los pasos demostrados en clase: conectar, crear cursor, ejecutar sentencias con parámetros posicionales (`?, ?`) para prevenir inyección SQL, confirmar cambios con `commit()`, y cerrar la conexión.

Diseño Preliminar del Sistema

7.5 Interfaz Gráfica con Tkinter

8. Metodología de Desarrollo

Incremento	Semana	Entregable	Funcionalidades
Incremento 1 Estructura Base	Semana 12 (6-10 jul)	Anteproyecto + Estructura del código	Definición de clases del dominio (Persona, Estudiante, Docente, Socio, Libro, Préstamo, Reserva). Implementación de encapsulamiento, herencia y polimorfismo. Estructuras de datos (LinkedList, Queue, Stack). Creación del repositorio GitHub y estructura de carpetas. Avance documental 50% (Tarea 12).
Incremento 2 Funcionalidades Core	Semana 13 (13-17 jul)	Avance funcional 60%	Base de datos SQLite con tablas relacionales y CRUD completo. Conexión de la GUI con la base de datos (formularios funcionales). Implementación de préstamos, devoluciones y reservas. Algoritmos de ordenamiento y búsqueda integrados al sistema. Completar documento de especificación.
Incremento 3 Integración Final	Semana 14 (20-jul)	Entrega final	Integración de todos los módulos. Pruebas funcionales completas. Manual de usuario con capturas de pantalla. Generación del archivo .db poblado. Preparación de la defensa oral.

Herramientas y Tecnologías

Las herramientas utilizadas para el desarrollo del proyecto fueron: Python 3.11 como lenguaje de programación principal, Visual Studio Code como editor de código con la extensión Python (Microsoft) y GitLens para el control de versiones, Git para Windows y GitHub como plataforma de control de versiones colaborativa, SQLite como motor de base de datos relacional embebido, DB Browser for SQLite para la administración visual de la base de datos, y Tkinter como librería de interfaz gráfica.

Para el desarrollo del Sistema de Gestión de Biblioteca Universitaria se seleccionó el modelo de proceso incremental, debido a que permite entregar funcionalidades completas y verificables en cada incremento, facilitando la retroalimentación temprana y la corrección de errores antes de avanzar a la siguiente etapa. El proyecto se dividió en tres incrementos distribuidos a lo largo de las tres semanas de trabajo (semanas 12, 13 y 14 del cronograma académico).

Para la interfaz gráfica del sistema se utiliza Tkinter, la librería estándar de Python para interfaces de escritorio. Tal como el Ing. Bryan Vélez explicó en la clase de la semana 12, Tkinter permite diseñar ventanas, botones, campos de texto, listas desplegables y demás componentes visuales mediante líneas de código Python. La importación se realiza con la instrucción 'import tkinter as tk', creando un alias para facilitar su uso. El docente enfatizó que la prioridad es la funcionalidad del sistema antes que la estética visual, indicando que una interfaz sencilla pero funcional es suficiente para la evaluación del proyecto.

9.1 Diagrama de Clases UML

El sistema sigue una arquitectura basada en POO con las siguientes clases principales:

Persona (clase abstracta): contiene los atributos comunes `__nombre`, `__identificacion`, `__telefono`, `__correo` y el método abstracto `tipo_socio()` que lanza `NotImplementedError`. Estudiante y Docente heredan de Persona e implementan `tipo_socio()` retornando 'Estudiante' y 'Docente' respectivamente.

Socio (clase envoltorio): recibe una referencia a Persona en su constructor y delega polimórficamente. Contiene además el método `get_limite_prestamo()` que retorna 3 para estudiantes y 5 para docentes.

Libro: modela un ejemplar bibliográfico con atributos como `__titulo`, `__autor`, `__isbn`, `__anio`, `__categoria`, `__ejemplares` y `__disponibles`.

Prestamo: registra un préstamo asociando un Socio, un Libro, una fecha de préstamo y una fecha de devolución.

Reserva: representa una solicitud de reserva cuando un libro no está disponible, manteniendo una cola de espera FIFO.

LinkedList (lista enlazada): estructura de nodos que almacena el catálogo de libros. Queue y Stack se implementan como envoltorios sobre LinkedList con operaciones específicas (enqueue/dequeue para la cola, push/pop para la pila).

9.2 Modelo Entidad-Relación de la Base de Datos

La base de datos relacional está compuesta por las siguientes tablas:

socios: almacena los datos de cada socio (id, nombre, identificación, tipo, telefono, correo). La columna tipo distingue entre Estudiante y Docente.

libros: contiene el catálogo bibliográfico (id, titulo, autor, isbn, año, categoria, ejemplares, disponibles).

prestamos: registra los préstamos activos e históricos (id, socio_id FK, libro_id FK, fecha_prestamo, fecha_devolucion, estado).

reservas: gestiona la cola de espera (id, socio_id FK, libro_id FK, fecha_reserva, posicion).

Las tablas socios y libros se relacionan con prestamos y reservas mediante claves foráneas (FK), estableciendo una relación de uno a muchos: un socio puede tener varios préstamos, y un libro puede estar involucrado en varios préstamos a lo largo del tiempo.

10. Distribución del Trabajo

El presente documento fue elaborado de forma colaborativa por los integrantes del Grupo #1, organizados con roles técnicos específicos que reflejan las responsabilidades reales del desarrollo del sistema. A continuación, se detalla la asignación de roles y responsabilidades de cada miembro:

Integrante	Rol	Responsabilidades principales
Henry Pazmiño	? Líder / Integrador	Coordinar el repositorio en GitHub, integrar el código de todos los miembros, asegurar el funcionamiento conjunto del sistema, edición final del documento de especificación
Jodie Parrales	? Desarrolladora GUI	Implementar la interfaz gráfica con Tkinter (formularios de socios, catálogo, préstamos, devoluciones, reservas). Conectar cada ventana con la base de datos
Cindy Ayoví	Desarrolladora BD	Diseñar y poblar la base de datos SQLite (biblioteca.db). Implementar operaciones CRUD completas de socios y libros. Garantizar la integridad referencial
Cesar Gonzales	Desarrollador Lógica/Negocio	Implementar la lógica de préstamos y devoluciones, reglas de negocio (límite de libros por tipo de socio), integración de algoritmos de ordenamiento y búsqueda al sistema
Mayra Vera	? Documentación y Pruebas	Redactar el documento de especificación, el manual de usuario con capturas de pantalla, realizar pruebas unitarias del sistema y verificar el cumplimiento de requisitos

11. Repositorio y Avance en GitHub

El código fuente del proyecto se encuentra alojado en un repositorio público de GitHub, donde se puede verificar el historial de commits y la estructura del sistema:

Repositorio: <https://github.com/josuepaz80-usitee/sistema-biblioteca-prog2>

A la fecha de esta entrega, el repositorio cuenta con los siguientes commits que reflejan el avance del proyecto:

#	Fecha	Mensaje del commit	Autor
1	7-jul	Initial commit	Grupo #1
2	7-jul	feat: estructura inicial del Sistema de Gestion de Biblioteca Universitaria	Grupo #1
3	7-jul	refactor: estilo nivel 3er semestre segun lo ensenado por Ing. Bryan Velez + DECLARATORIA	Grupo #1
4	7-jul	fix: DECLARATORIA actualizada	Grupo #1

		tras revision completa del material del semestre	
5	7-jul	fix: referencias genericas sin nombres individuales en DECLARATORIA	Grupo #1
6	11-jul	docs: agregar documentacion del proyecto (especificacion, manual, Gantt, README con roles)	Henry Pazmiño
7	11-jul	docs: especificacion visible en markdown + creditos de autoría por integrante en cada archivo	Henry Pazmiño
8	11-jul	fix: formato limpio del credito de Mayra en manual-usuario.md	Henry Pazmiño
9	11-jul	fix: actualizar historial de commits y DECLARATORIA (Tkinter + incremental)	Henry Pazmiño
10	11-jul	docs: actualizar DOCX y PDF con Metodologia de Desarrollo + commits hasta #9	Henry Pazmiño

Estructura actual del repositorio:

- src/models/: clases del dominio (Persona, Estudiante, Docente, Socio, Libro, Prestamo, Reserva)
- src/data_structures/: lista enlazada simple (LinkedList), cola FIFO (Queue), pila LIFO (Stack)
- src/algorithms/: ordenamiento (burbuja, inserción, merge sort, quick sort) y búsqueda (lineal y binaria)
- src/database/: conexión SQLite y operaciones CRUD (db_manager.py, socio_repo.py, libro_repo.py)
- src/gui/: interfaz gráfica con Tkinter (app.py)
- tests/: pruebas unitarias del modelo
- DECLARATORIA.md: declaración de uso de IA e investigación con referencias APA

El grupo decidió utilizar GitHub como plataforma de control de versiones para el desarrollo colaborativo del proyecto. Para ello, cada integrante instaló en su computadora personal el siguiente stack tecnológico durante una reunión de coordinación: Visual Studio Code como editor de código, la extensión de Python (Microsoft) para soporte del lenguaje, la extensión GitLens para visualizar el historial de cambios, Git para Windows como sistema de control de versiones local, y Python 3.11+ como lenguaje de programación. El repositorio fue creado en GitHub y todos los miembros fueron agregados como colaboradores, permitiendo que cada uno pueda realizar commits y mantener el código sincronizado.

Cabe señalar que el proyecto final contempla tres entregables: (1) el Documento de Especificación del proyecto (el presente avance del 50% que se completa en la semana 14), (2) el Manual de Usuario del sistema con capturas de la interfaz funcionando, y (3) la Implementación completa del sistema con el código fuente funcional, la base de datos en SQLite y la interfaz gráfica con Tkinter. Los tres entregables se presentarán en la semana 14.

Este plan fue elaborado en base a las instrucciones del Ing. Bryan Vélez, quien estableció que el avance funcional mínimo del 60% debe estar listo para la semana 13, y la entrega completa (documento 100%, código funcional, BD poblada, manual de usuario) debe finalizar el lunes 20 de julio.

Para completar la entrega final del proyecto en la semana 14 (lunes 20 de julio), el equipo ha definido el siguiente plan de trabajo con tareas asignadas y prioridades:

12. Plan de Trabajo — Próximos Pasos

A continuación se presenta el cronograma visual del proyecto (diagrama de Gantt), el cual abarca desde el inicio del proyecto el 7 de julio (semana 12) hasta la entrega final el 20 de julio (semana 14). Las barras de colores representan las tareas de cada integrante, y los diamantes rojos indican los commits con sentido que deben reflejar el avance real en el repositorio de GitHub.

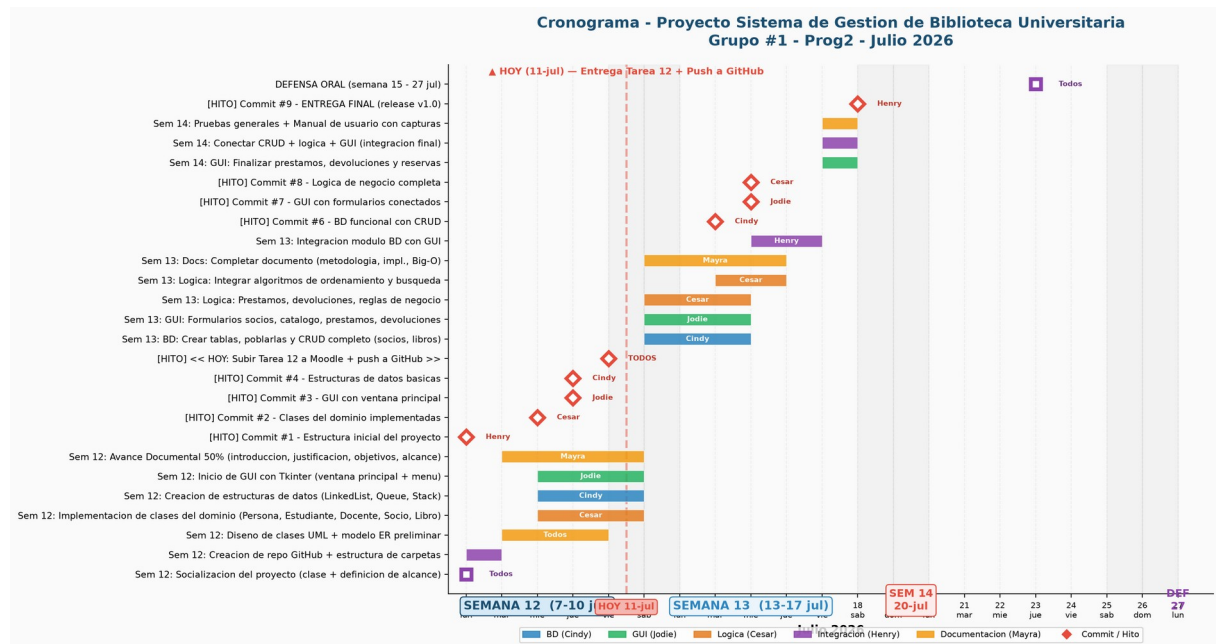


Figura 1. Cronograma del proyecto (diagrama de Gantt) — Julio 2026

Tarea	Responsable	Prioridad
GUI funcional — conectar formularios de socios, libros, préstamos y reservas con la base de datos	Jodie	Alta
BD poblada — crear biblioteca.db con datos de prueba (10+ socios, 15+ libros)	Cindy	Alta
Lógica de préstamos, devoluciones, reservas y reglas de negocio implementada	Cesar	Alta
Completar el documento (metodología, implementación, Big-O, conclusiones, bibliografía)	Mayra	Media
Manual de usuario con capturas de la interfaz funcionando	Mayra	Media
Integración de todos los módulos y verificación de funcionamiento conjunto	Henry	Alta
Ensayar defensa oral — todos los integrantes deben poder explicar todo el código	Todos	Semana 15